CPSC 449 Lab 12: Context-Aware Movie Chat with Gemini

Overview

In this lab, you will extend the **Movie Manager** with a Gemini-powered chat agent. The starter application uses Node.js, Express, Socket.IO, browser JavaScript, and the Google Gen AI SDK. Messages containing `@gemini` are sent from the browser to the Express server, forwarded to Gemini, and streamed back to connected clients.

The starter currently sends only the newest user message to the model. It does not load the application's movie data, reviews, user preferences, or earlier conversation turns. As a result, its responses are general rather than grounded in the application's data or personalized to the current conversation.

Your main task is to connect the agent to an application database, construct relevant context, preserve conversation history, and evaluate whether its answers accurately reflect the source data.

Learning Objectives

By the end of this lab, you should be able to:

- Call a generative AI model from a Node.js backend.
- Stream model output through a Socket.IO connection.
- Keep an API key on the server and outside source control.
- Explain the difference between model knowledge and application-provided context.
- Retrieve movie, review, preference, and conversation data from a database.
- Construct multi-turn `user` and `model` conversation history.
- Ground responses in data supplied by the application.
- Limit context to relevant information and account for context-window cost.
- Evaluate generated answers against authoritative database records.
- Recognize hallucinations, unsupported claims, and stale context.

Before You Start

1. Install Node.js and npm if they are not already installed.
2. Create or use a Google account that can access Google AI Studio.
3. Prepare the movie, review, and conversation data from earlier labs, or choose another database containing equivalent data.
4. Review the provider's current model availability, pricing, quotas, and data-use terms before sending any sensitive information.

Verify Node.js and npm from a terminal:

```
node -v
npm -v
```

Never commit a Gemini API key or expose it in browser JavaScript. The browser should communicate with the Express backend, and only the backend should call Gemini.

Project Structure

```
.
|-- images/
|   ...
|-- public/
|   |-- details.html
|   |-- details.js
|   |-- index.html
|   |-- manager.html
|   |-- manager.js
|   |-- script.js
|   `-- styles.css
|-- .env.sample
|-- .gitignore
|-- package.json
|-- package-lock.json
`-- server.js
```

- `server.js` configures Express and Socket.IO, calls Gemini, and provides the starter movie routes.
- `public/index.html` contains the regular movie search and Gemini chat interface.
- `public/script.js` sends user messages and renders streamed agent or manager replies.
- `public/manager.html` and `public/manager.js` provide the manager dashboard and live-chat view.
- `public/details.html` and `public/details.js` provide the starter movie-details view.
- `.env.sample` documents the required `GEMINI_API_KEY` variable without containing a real key.
- `images/` contains the Google AI Studio and Gemini screenshots used in this README.

Request and Context Flow

```
Browser sends userMessage over Socket.IO
                |
                v
Express detects the @gemini mention
                |
                v
Load relevant database records
  - supported movies and genres
  - reviews and ratings
  - user preferences
  - prior conversation turns
                |
                v
Build ordered Gemini contents/history
                |
                v
Gemini generates a grounded response
```

```
                  |
                  v
Express emits agentReply to the browser
                  |
                  v
Save the new user and model turns
```

The database remains the source of truth. Gemini receives selected records as context and summarizes or reasons over them; it should not invent records that are absent from that context.

Setup and Running the App

1. Open a terminal in the project directory.
2. Install the dependencies:

   ```
   npm install
   ```

3. Copy `.env.sample` to `.env`.
4. Replace the placeholder with your own API key:

   ```
   GEMINI_API_KEY=your_api_key_here
   ```

5. Add any database connection variables required by your implementation. Never place credentials in browser files.
6. Start the development server:

   ```
   npm run dev
   ```

7. Open the user interface:

   ```
   http://localhost:3000/index.html
   ```

8. Optionally open the manager interface in another browser session:

   ```
   http://localhost:3000/manager.html
   ```

Always load the pages through Express. Keep the server terminal and browser developer tools open while testing model requests, Socket.IO events, and database queries.

Application Routes

Method	Route	Starter behavior
`GET`	`/movies`	Returns the complete in-memory movie list.
`GET`	`/movies/genre/:genre`	Returns movies matching a genre.
`GET`	`/movies/search/:name`	Returns a placeholder result until the search TODO is completed.

The starter has no database package or persistence layer yet. Add the appropriate client library and connection code for the database you choose.

Real-Time Application Events

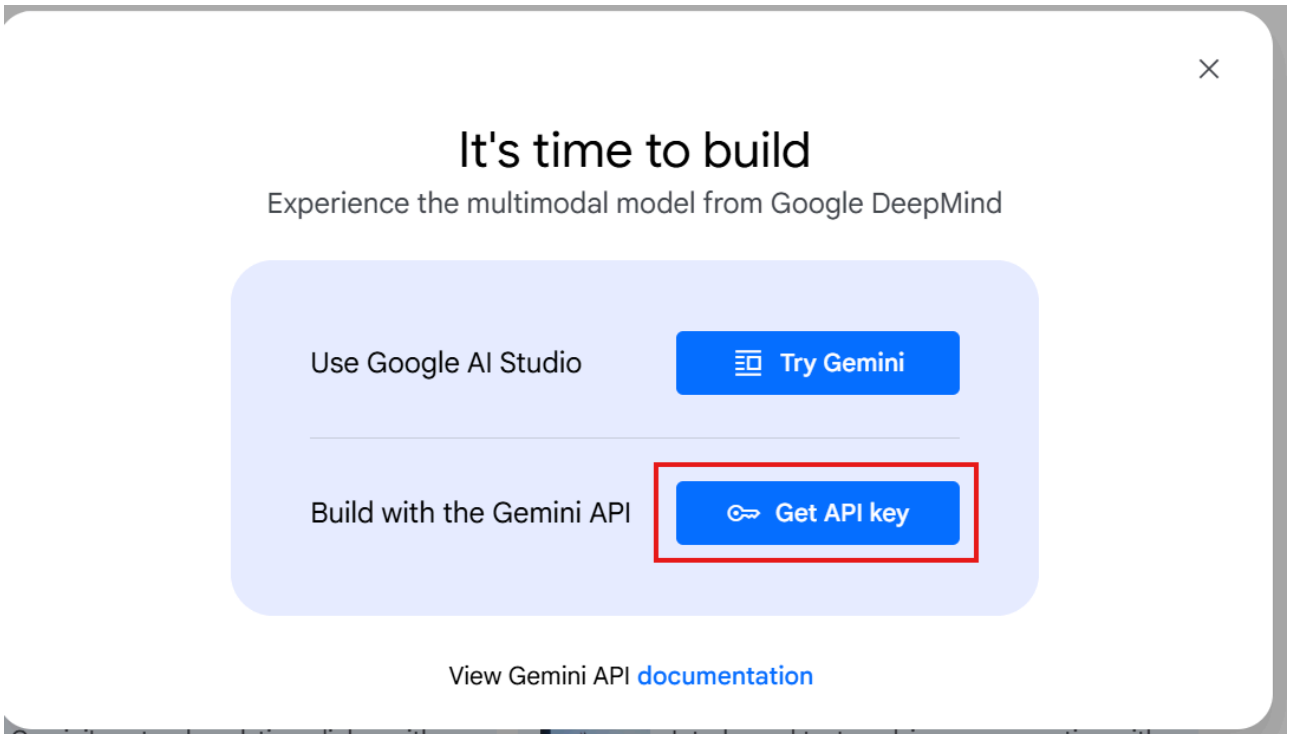
Direction	Event	Expected behavior
User client to server	`userMessage`	Send a chat message to Express.
Server to manager client	`userIncomingMessage`	Display the user's message in the manager dashboard.
Server to connected clients	`agentReply`	Display Gemini's generated response.
Manager client to server	`managerReply`	Send a human manager's response.
Server to other clients	`managerReply`	Display the manager's response.

Only messages containing `@gemini`, matched case-insensitively, trigger a Gemini request in the starter.

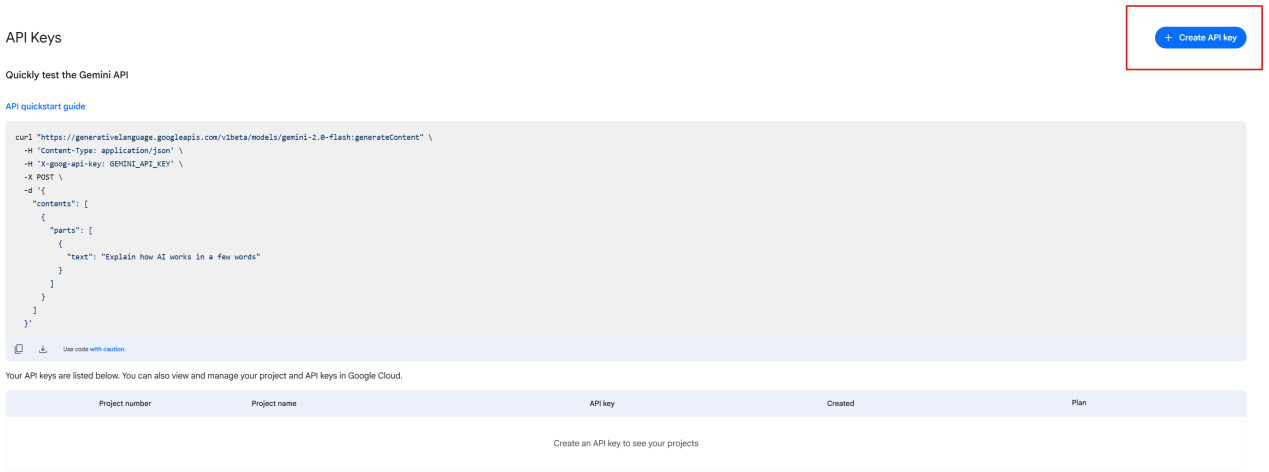
Google AI Studio Setup

Use the following screenshots as a reference. Google AI Studio, available models, and account options can change, so the current interface may differ.

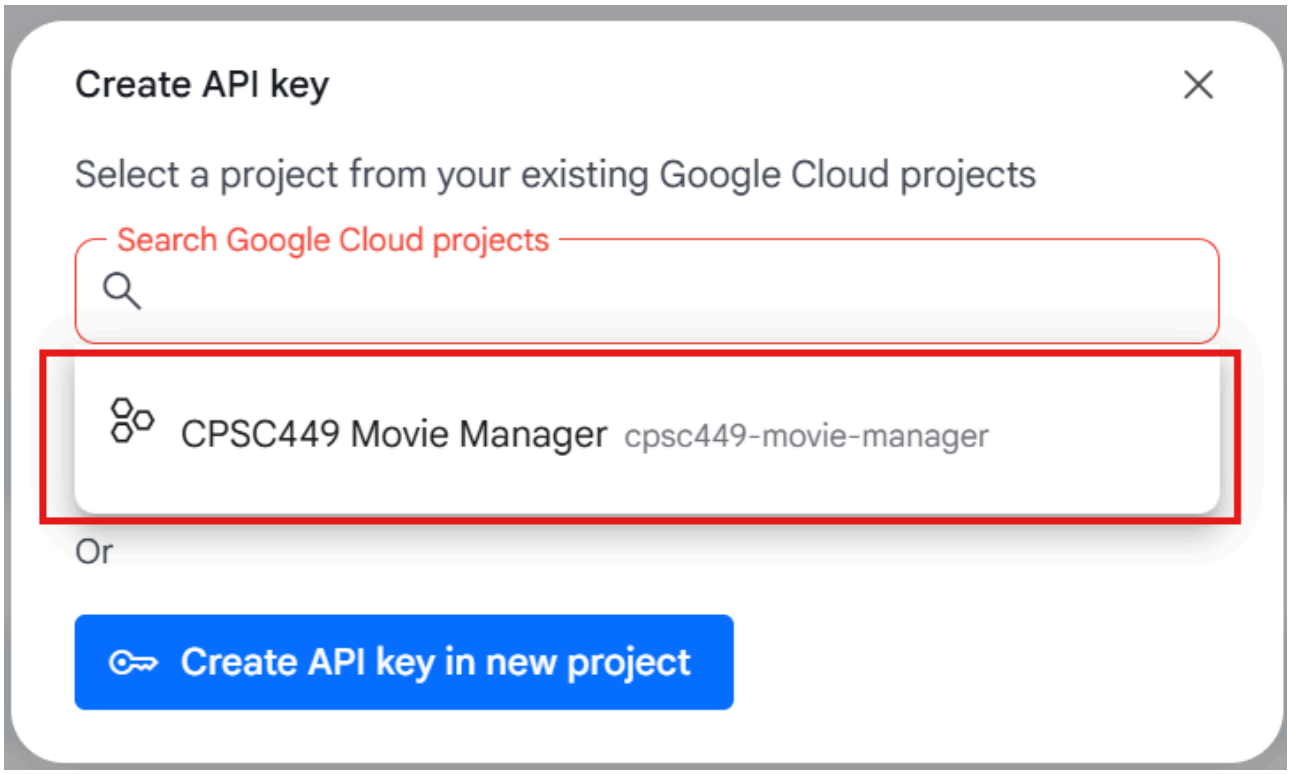
1. Open Google AI Studio and sign in.



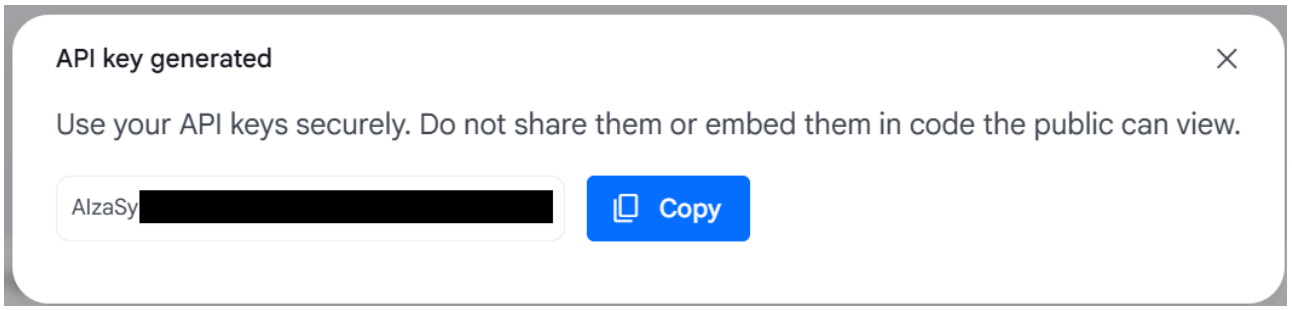
2. Open the API-key section and choose the option to create a key.



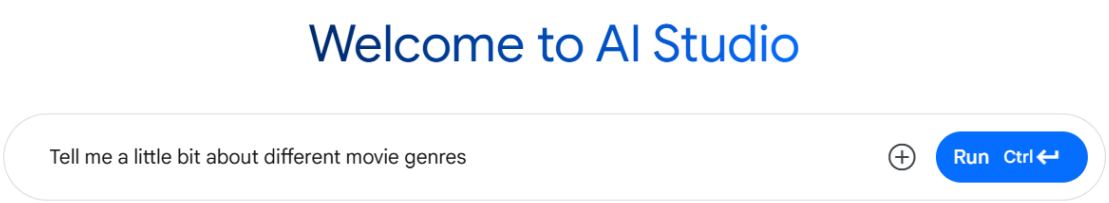
3. Select an existing Google Cloud project or create a project for this lab.



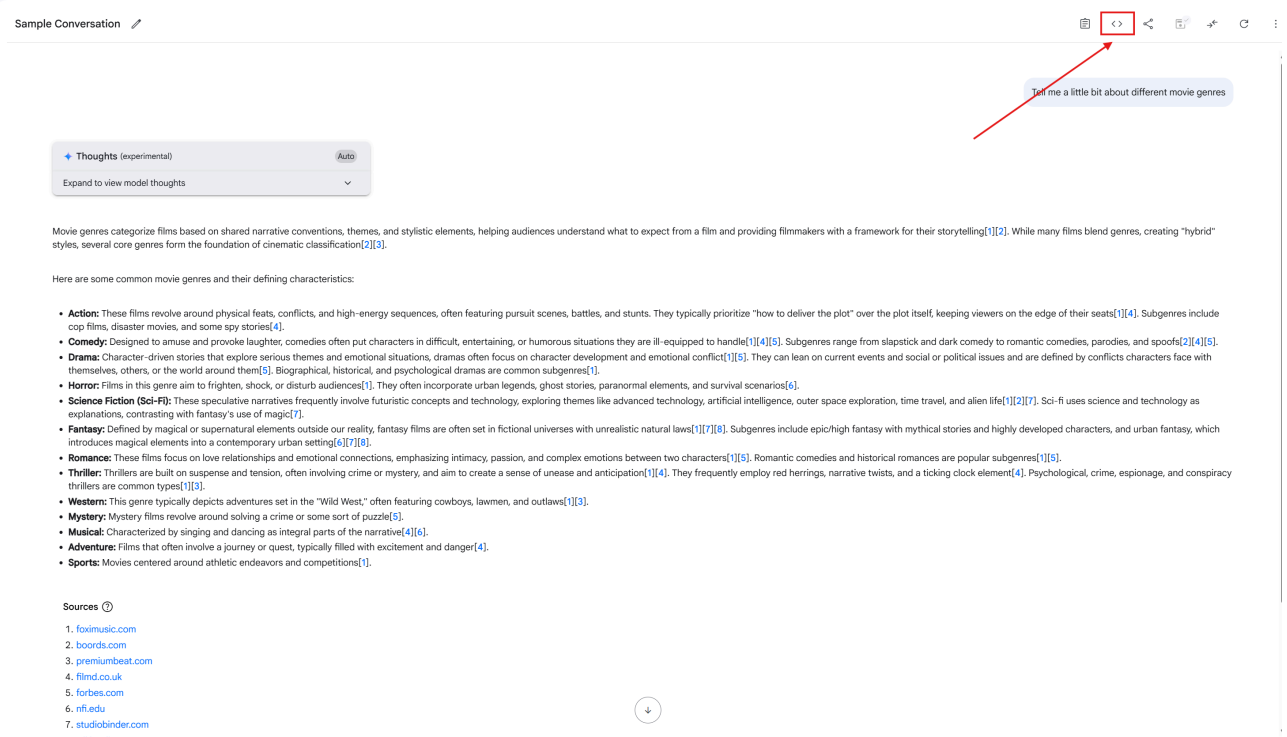
4. Generate the API key.



5. Copy the key once and store it only in `.env` or an approved secret manager.



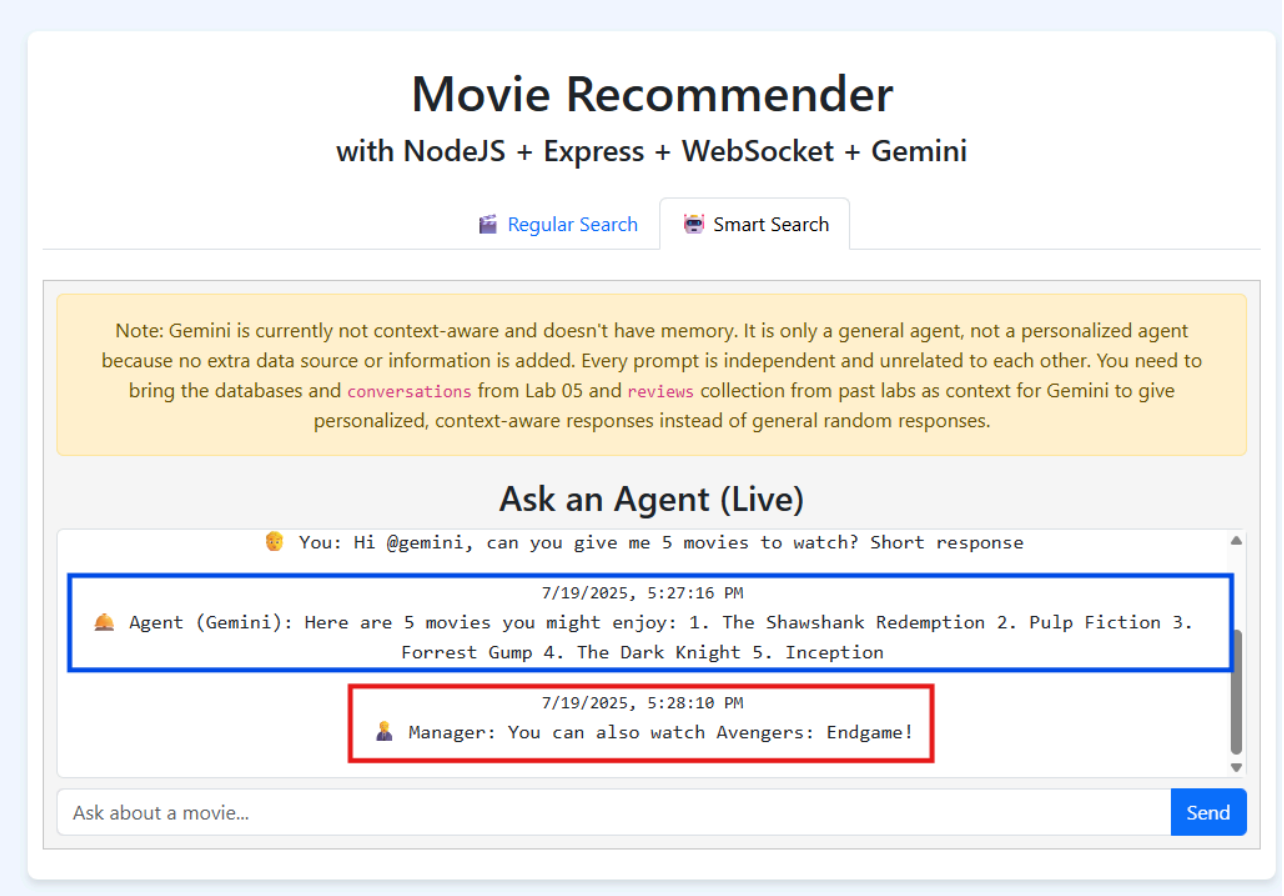
6. Create a sample multi-turn conversation in AI Studio, then export it with prompt history enabled.



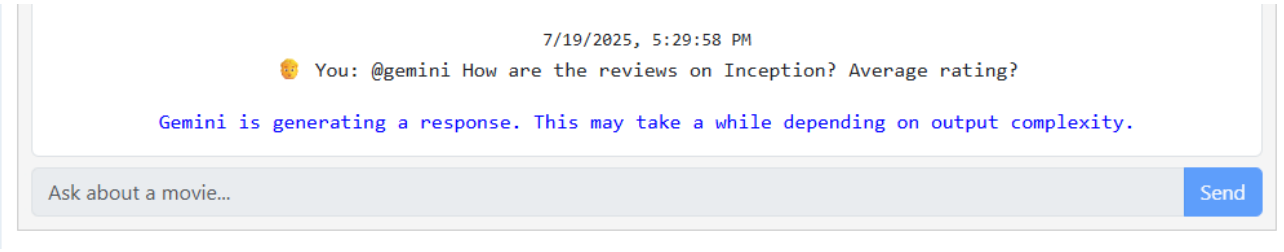
7. Inspect how alternating user and model turns are represented in the exported history.



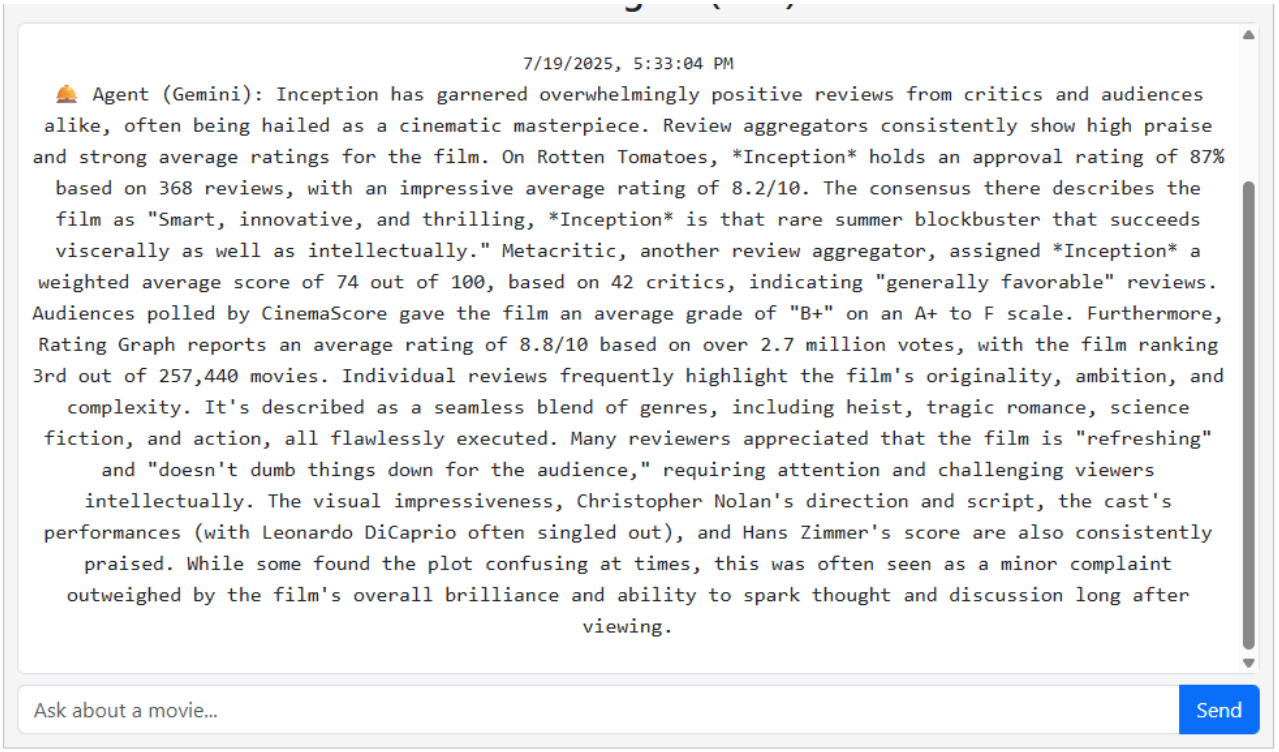
8. Review the model and generation configuration before copying an example into the application.



9. Test a prompt in AI Studio before reproducing the request through the application.



10. Confirm that the server-side integration returns a response through the movie-chat interface.



Model names and access can change over time. The starter currently selects `gemini-2.5-flash`; if that model is unavailable to your project, choose a currently supported model and document the change.

Building Grounded Context

The starter's `contents` array contains only the current message:

```
const contents = [
  {
    role: 'user',
    parts: [{ text: msg }]
  }
];
```

For this lab, build the request from relevant application data. A useful order is:

1. Application instructions that define the agent's task and prohibit unsupported claims.
2. Selected movie, genre, rating, and review records from the database.
3. User preferences relevant to the current request.
4. Earlier `user` and `model` turns in chronological order.
5. The newest user message.

Do not send the entire database with every prompt. Retrieve only records relevant to the question, limit the number and length of reviews, and record which source data was used. Smaller, focused context is easier to inspect and usually consumes fewer tokens.

Conversation history must include both sides of the exchange. Saving only user messages prevents the model from seeing what it previously answered. Keep histories isolated by user or conversation ID so one user's data is not disclosed to another.

The starter also enables Google Search as a model tool. Search results are different from your private application context. For assignment testing, make it possible to demonstrate that an answer came from the database rather than from web search or general model knowledge.

Student TODOs

Complete the TODOs in order. Verify each step before moving to the next one.

TODO 1: Run and Inspect the Starter

1. Add `GEMINI_API_KEY` to a local `.env` file.
2. Confirm that `.env` is ignored by Git.
3. Start the server and open `index.html` through Express.
4. Send a normal chat message and observe the Socket.IO behavior.
5. Send a message containing `@gemini` and confirm that a model response appears.
6. Inspect `server.js` and identify why each Gemini request is currently independent.

TODO 2: Connect the Application Database

1. Choose MongoDB or another database containing your movie, review, and conversation data.
2. Install the appropriate server-side database package.
3. Store the database connection value in `.env`.
4. Connect before handling requests that require database context.
5. Add focused queries for supported genres, matching movies, reviews, preferences, and conversation turns.
6. Handle connection and query failures without sending secrets to the browser.

TODO 3: Ground Responses in Movie Data

1. Determine which database records are relevant to the current prompt.
2. Convert those records into compact, clearly labeled text or structured parts.
3. Add instructions telling the model to answer from the supplied records and acknowledge missing information.
4. Append the current user message after the database context.
5. Test questions about supported genres, specific recommendations, and review summaries.
6. Compare every factual claim with the source documents.

TODO 4: Add Conversation Memory

1. Assign or retrieve a conversation ID for each chat session.
2. Load prior user and model messages in chronological order.
3. Convert stored turns into valid Gemini `user` and `model` history entries.
4. Include the history before the newest message or use the SDK's supported multi-turn chat mechanism.
5. Save the new user message and completed model reply.
6. Limit retained history by count, age, relevance, or token budget.
7. Keep one user's conversation history separate from every other user's history.

Test memory by sharing a movie or genre preference, continuing the conversation, and later asking Gemini to recall it.

TODO 5: Evaluate and Document

1. Ask Gemini which genres the database supports.
2. Ask for a specific recommendation from the stored movie list.
3. Ask for an overall summary, status, and rating based on stored reviews.
4. Capture each prompt, response, and the database records used as evidence.
5. Identify every accurate, inaccurate, unsupported, or omitted claim.
6. Report an accuracy measure and explain how you calculated it.
7. Discuss whether changing the context improved the answer.

Do not claim that a response is grounded merely because it sounds correct. The evidence must show that the relevant data was retrieved and included in the request.

Completion Checklist

- [ ] The application runs locally and calls Gemini from the backend.
- [ ] API keys and database credentials remain outside source control.
- [ ] Movie, review, and conversation data load successfully.
- [ ] Prompts include relevant database context and ordered chat history.
- [ ] Conversation histories remain isolated by user or session.
- [ ] Responses accurately reflect the stored genres, movies, and reviews.
- [ ] New chat turns persist and reload correctly.
- [ ] Errors are handled without breaking the chat interface.

Submission Requirements

Submit the items required by your instructor, including:

1. A screenshot of the database records used as model context.
2. A prompt and response covering the genres supported by the database.
3. A prompt and response requesting a supported movie recommendation.
4. A prompt and response summarizing stored movie reviews and ratings.
5. Evidence that a later prompt correctly uses an earlier conversation detail.
6. An accuracy report comparing generated claims with the database.
7. A brief description of how context is retrieved, limited, ordered, and saved.
8. Your completed project files in the correct repository and branch.

Before submitting, confirm that screenshots are readable and contain no API key, connection string, private user information, or other secret. Do not commit `.env` or `node_modules/`.

Extra Credit (+5 points)

With instructor approval, extend the project with one of the following:

- Generate an image related to a supported movie or recommendation. Follow the Gemini Image Generation API documentation for JavaScript.
- Allow a user to upload media or another supported file type for analysis. Follow the Gemini Files API media-upload documentation for JavaScript.
- Add retrieval that ranks records by relevance before constructing context.

- Add source labels or citations to database-grounded responses.

Keep API calls on the server, validate uploaded content, enforce size limits, and document any additional model, storage, or billing requirements.

Troubleshooting

The server does not start

- Run `npm install` and try `npm run dev` again.
- Confirm that `.env` exists and contains `GEMINI_API_KEY`.
- Confirm that port `3000` is not already in use.
- Read the first meaningful error in the server terminal.

Gemini rejects the request

- Confirm that the API key is valid and belongs to the intended project.
- Check current model availability and API-key restrictions.
- Review the HTTP status, provider error message, and current quota.
- Confirm that the request structure matches the installed `@google/genai` version.
- Rotate the key immediately if it was committed or exposed.

Gemini does not answer

- Include `@gemini` in the message.
- Check the browser console for a Socket.IO connection error.
- Check the server terminal for the `userMessage` event and Gemini error.
- Confirm that chat controls are re-enabled in the error path.

Gemini forgets earlier messages

- Confirm that prior user and model turns are loaded before each new request.
- Sort persisted messages chronologically.
- Use consistent `user` and `model` roles.
- Confirm that each completed reply is saved before the next request.
- Check whether history limits removed the relevant turn.

Gemini invents movies or review details

- Reduce unrelated context and clearly label authoritative records.
- Tell the model to use only the supplied records for database-specific claims.
- Ask it to state when the database lacks an answer.
- Verify that the query returned the intended movie and reviews.
- Separate web-search results from private database context during evaluation.

Responses become slow or expensive

- Retrieve fewer and more relevant records.
- Limit conversation history and review length.
- Summarize older history while preserving important facts.

- Avoid resending duplicate context.
- Monitor current token usage, rate limits, and billing settings.

References

- Google Gen AI SDK and Gemini API quickstart
- Using Gemini API keys securely
- Gemini text generation and multi-turn conversations
- Gemini API rate limits
- Gemini image generation
- Gemini file API